Católica
Universidade Católica de Brasília

Pró-Reitoria Acadêmica
Escola de Educação, Tecnologia e Comunicação
Curso de Bacharelado em Engenharia de Software
Trabalho da Disciplina de Análise e Projeto de Software

Cashlens - MVP de Gestão Financeira

Documento de Arquitetura do Sistema

Versão 1.0

Autores: João Marcos Azevedo Cruz e João Pedro Tavares Teixeira

Orientador: Alexandre Silva dos Santos

REVISÕES DO DOCUMENTO

Este documento registra as revisões relacionadas ao Documento de Arquitetura do Sistema do projeto CashLens. Seu objetivo é manter o histórico de alterações realizadas ao longo da evolução da documentação.

Versão	Data	Autores	Observações
1.0	13/04/2026	Equipe CashLens	Criação da versão inicial do documento de arquitetura

SUMÁRIO

1. INTRODUÇÃO **4**
1.1 Finalidade 4
1.2 Escopo 4

2. DESCRIÇÃO GERAL DA ARQUITETURA **4**
2.1 Estilo Arquitetural Adotado 5

3. PRINCIPAIS COMPONENTES E RESPONSABILIDADES **6**

4. DIAGRAMA DE ARQUITETURA **7**
4.1 Diagrama de Arquitetura em Camadas 7

5. TECNOLOGIAS PREVISTAS **7**

6. JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS **9**

7. CONCLUSÃO **10**

1. INTRODUÇÃO

Este documento apresenta a arquitetura do sistema CashLens, descrevendo como a solução foi estruturada para suportar o MVP proposto na disciplina de Análise e Projeto de Software. A arquitetura definida busca atender às funcionalidades centrais do sistema, como cadastro e autenticação de usuários, ingestão de arquivos CSV, processamento de dados financeiros, consultas analíticas e persistência das informações.

A documentação de arquitetura complementa os artefatos já produzidos, como o Documento de Visão, o Documento de Requisitos de Software e a Especificação de Casos de Uso, concentrando-se na organização técnica da solução, nos componentes que a compõem e nas decisões arquiteturais adotadas para sua implementação.

1.1 Finalidade

A finalidade deste documento é descrever a arquitetura do CashLens de forma clara e compreensível para diferentes stakeholders, incluindo equipe técnica, docentes e demais envolvidos no projeto, permitindo o entendimento da estrutura do sistema, do fluxo entre seus componentes e das tecnologias previstas para sua construção.

1.2 Escopo

O escopo desta documentação abrange a arquitetura do MVP do CashLens, contemplando a aplicação web em **React,** a camada de identidade e acesso com **Keycloak**, o backend em **Spring Boot**, a proteção das APIs com Spring Security, a persistência em **PostgreSQL** e o fluxo de ingestão de dados por arquivos CSV.

Também fazem parte deste escopo os mecanismos de comunicação entre front-end e backend por meio de APIs REST, o uso de token JWT para autenticação das requisições e a separação da solução em camadas com responsabilidades técnicas bem definidas.

2. DESCRIÇÃO GERAL DA ARQUITETURA

O CashLens adota uma arquitetura web baseada no modelo cliente-servidor, na qual o front-end e o backend desempenham papéis distintos e se comunicam por meio de APIs REST. No lado cliente, a aplicação React executa no navegador e é responsável pela navegação entre telas, exibição dos painéis analíticos, interação com o usuário e envio das requisições ao servidor. No lado servidor, a aplicação Spring Boot concentra o processamento das requisições, a aplicação das regras de negócio, a ingestão dos arquivos CSV, a validação dos dados recebidos e a persistência das informações no banco PostgreSQL.

A arquitetura também incorpora uma camada específica de identidade e acesso,

representada pelo Keycloak. Essa camada é responsável pela autenticação do usuário e pela emissão do token JWT utilizado no acesso às APIs do sistema. Após autenticado, o usuário interage com a interface web, que passa a consumir os endpoints protegidos do backend por meio de Bearer Token. No servidor, o Spring Security realiza a validação do token antes de permitir que a requisição alcance os controllers REST e os demais componentes internos da aplicação.

Do ponto de vista estrutural, a solução foi organizada em camadas, separando apresentação, identidade e acesso, aplicação, negócio e dados. No front-end, essa organização aparece na divisão entre rotas, estado da aplicação, interface analítica e consumo de APIs. No backend, a estrutura é distribuída entre segurança, controllers, services, ingestão de CSV, persistência e banco de dados. Essa separação reduz o acoplamento entre responsabilidades, melhora a manutenibilidade e favorece a evolução incremental do sistema.

Além da base principal da arquitetura, algumas tecnologias complementares apoiam a implementação. No front-end, React Router e Zustand contribuem para a navegação e o gerenciamento de estado. No backend, JPA facilita a persistência orientada a objetos, OpenAPI/Swagger apoia a documentação técnica dos endpoints e Flyway controla a evolução versionada do banco de dados.

2.1 Estilo Arquitetural Adotado

O estilo arquitetural do CashLens combina o modelo cliente-servidor com uma arquitetura em camadas, implementada em um backend monolítico modular. Essa combinação foi adotada para organizar o sistema de forma clara, mantendo a interface, a autenticação, o processamento e a persistência separados em responsabilidades técnicas específicas.

No modelo cliente-servidor, o cliente concentra a interação com o usuário e a apresentação das informações, enquanto o servidor centraliza as regras de negócio, a segurança das APIs, o processamento dos arquivos CSV e o acesso aos dados persistidos. Já a arquitetura em camadas organiza a solução em blocos com responsabilidades distintas, evitando que regras de negócio, autenticação, interface e persistência se misturem de forma inadequada.

No front-end, a camada de apresentação reúne as telas do sistema, o fluxo de navegação e o consumo das APIs protegidas. Na camada de identidade e acesso, o Keycloak gerencia a autenticação dos usuários e a emissão dos tokens JWT. No backend, o Spring Security atua como camada de proteção das rotas, seguido pelos controllers REST, que recebem as requisições e as encaminham para os services, onde estão concentradas as regras de negócio. A ingestão de CSV trata especificamente o processamento dos arquivos de entrada, enquanto

a camada de persistência realiza a comunicação com o PostgreSQL.

De forma resumida, a arquitetura adotada no CashLens pode ser caracterizada pelos seguintes aspectos:

- **Cliente-servidor:** o front-end em React concentra a interação com o usuário, enquanto o backend em Spring Boot centraliza o processamento das requisições, as regras de negócio e o acesso aos dados persistidos.
- **Arquitetura em camadas:** a solução foi organizada em apresentação, identidade e acesso, aplicação, negócio e dados, separando responsabilidades técnicas e reduzindo o acoplamento entre as partes do sistema.
- **Monólito modular:** No backend, a estrutura é distribuída entre segurança, aplicação e negócio, ingestão de CSV, persistência e banco de dados.
- **Componentização no front-end:** a interface foi organizada em componentes reutilizáveis, com separação entre navegação, estado da aplicação, consumo de APIs e telas analíticas.
- **Autenticação centralizada:** o Keycloak gerencia a autenticação dos usuários e a emissão de tokens JWT, enquanto o Spring Security protege as rotas do backend e valida os tokens recebidos nas requisições.

3. PRINCIPAIS COMPONENTES E RESPONSABILIDADES

Os principais componentes do CashLens e suas responsabilidades estão resumidos na tabela a seguir.

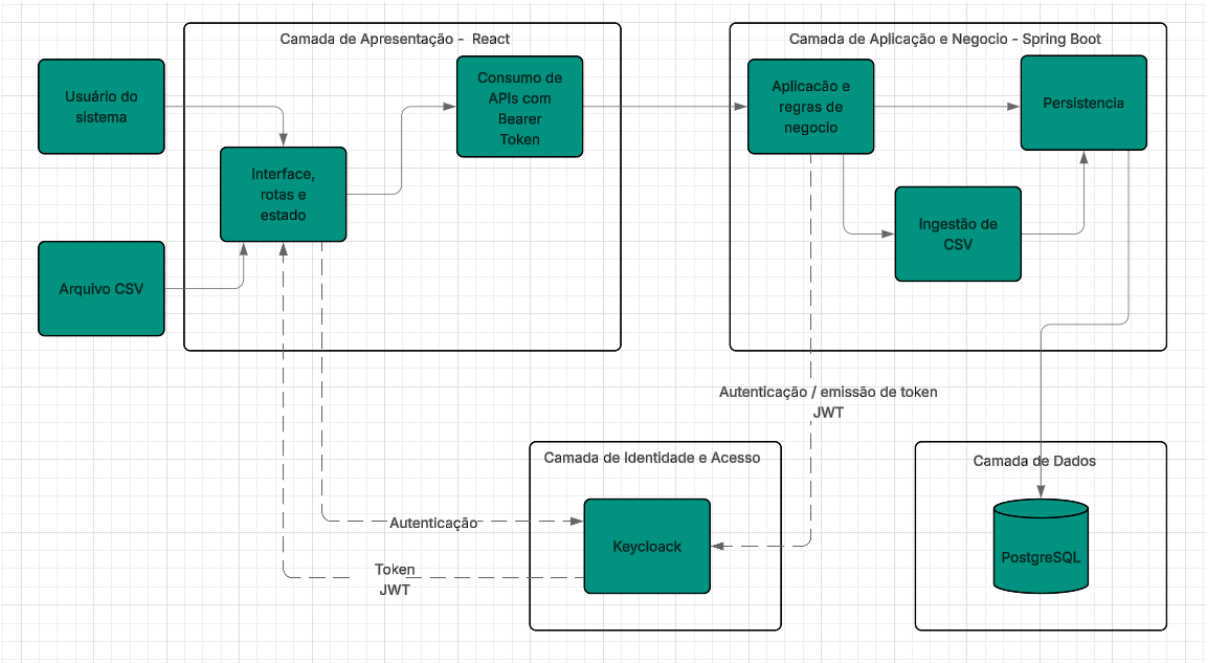
COMPONENTE	RESPONSABILIDADE	BASE TECNOLÓGICA
Camada de apresentação	Apresentar as telas do sistema, permitir a interação do usuário, controlar a navegação e exibir os recursos analíticos do MVP.	React
Consumo de APIs com Bearer Token	Realizar chamadas autenticadas ao backend para cadastro, login, importação, consultas analíticas e operações de manutenção dos dados.	React
Camada de identidade e acesso	Realizar a autenticação dos usuários e emitir os tokens JWT utilizados no acesso aos recursos protegidos da aplicação.	Keycloak
Segurança das APIs	Validar os tokens recebidos e proteger os endpoints do backend, garantindo que apenas requisições autenticadas sejam processadas.	Spring Security

Camada de aplicação e negócio	Receber requisições, coordenar o fluxo da aplicação, aplicar regras de negócio e integrar o processamento com ingestão e persistência.	Spring Boot
Ingestão de CSV	Receber arquivos CSV, validar sua estrutura e conteúdo, tratar inconsistências e preparar os dados para persistência.	Spring Boot
Persistência	Realizar o acesso aos dados da aplicação, incluindo registros financeiros, usuários, importações e informações de auditoria.	Spring Boot, JPA
Banco de dados	Armazenar de forma estruturada os dados financeiros, cadastrais, históricos e de auditoria do sistema.	PostgreSQL

4. DIAGRAMA DE ARQUITETURA

4.1 Diagrama de Arquitetura em Camadas

Para representar a arquitetura do CashLens foi adotado um diagrama de componentes em camadas. Esse tipo de diagrama é adequado porque evidencia os principais blocos da solução, o fluxo de comunicação entre usuário, interface web, autenticação, backend e banco de dados, além de destacar o papel da ingestão de arquivos CSV no contexto do MVP.



5. TECNOLOGIAS PREVISTAS

A arquitetura do CashLens utiliza um conjunto de tecnologias compatíveis com o escopo do MVP e com a organização em camadas definida para o sistema. Essas tecnologias

dão suporte à interface web, à autenticação dos usuários, ao processamento das regras de negócio, à ingestão de arquivos CSV e à persistência dos dados financeiros.

TECNOLOGIA	PAPEL NO SISTEMA	JUSTIFICATIVA
React	Implementação da camada de apresentação, incluindo interface, navegação e interação do usuário com os recursos analíticos do sistema.	Permite construir interfaces web modulares, reutilizáveis e adequadas à exibição de dashboards, filtros e telas analíticas.
Keycloak	Camada de identidade e acesso, responsável pela autenticação dos usuários e emissão de tokens JWT.	Centraliza o processo de autenticação e reduz o acoplamento entre segurança e lógica de negócio da aplicação.
Spring Boot	Implementação da camada de aplicação e negócio, incluindo APIs REST, regras de negócio, proteção dos endpoints, ingestão de CSV, integração com persistência e apoio à documentação e testes da aplicação.	Oferece uma base robusta para o desenvolvimento do backend, com suporte à criação de APIs REST, organização em camadas e integração com recursos do ecossistema Spring. Ferramentas como Spring Security, Spring Data JPA, Spring Test, OpenAPI/Swagger e Flyway ampliam o suporte à segurança, persistência, testes, documentação e evolução controlada do banco de dados.
PostgreSQL	Banco de dados relacional responsável pelo armazenamento das informações financeiras, cadastrais, históricas e de auditoria.	Garante integridade, consistência e bom suporte a relacionamentos entre entidades do domínio financeiro.
Docker / Docker Compose	Padronização do ambiente de desenvolvimento e execução integrada dos serviços da solução.	Facilitam a configuração do ambiente e reduzem inconsistências entre diferentes máquinas de desenvolvimento.

6. JUSTIFICATIVA DAS DECISÕES ARQUITETURAIS

As principais decisões arquiteturais do CashLens foram definidas com foco em organização estrutural, segurança, manutenibilidade e aderência ao escopo funcional do MVP.

DECISÃO	JUSTIFICATIVA	BENEFÍCIO ESPERADO
Arquitetura cliente-servidor	O sistema foi estruturado com separação entre a aplicação web, responsável pela interação com o usuário, e o backend, responsável pelo processamento das regras do sistema e acesso aos dados.	Facilita a divisão de responsabilidades, melhora a compreensão da solução e permite evolução mais organizada entre interface e servidor.
Arquitetura em camadas	A solução foi organizada em apresentação, identidade e acesso, aplicação e negócio, persistência e dados, evitando que autenticação, interface, regras de negócio e acesso ao banco fiquem acoplados.	Melhora a manutenção, a testabilidade e a clareza estrutural do sistema.
React como base da camada de apresentação	O sistema exige uma interface web com navegação entre telas, componentes reutilizáveis e visualização de informações analíticas, como dashboards, DRE, timeline de caixa e gráficos.	Melhora a organização da interface, favorece a reutilização de componentes e facilita a evolução das telas do sistema.
Spring Boot como backend central	O backend foi concentrado em uma única aplicação responsável por expor as APIs, processar regras de negócio, integrar a ingestão de CSV e coordenar a persistência dos dados.	Reduz a complexidade arquitetural do MVP e mantém a lógica principal do sistema em um ponto centralizado e mais fácil de manter.
Keycloak como camada de identidade e acesso	A autenticação dos usuários e a emissão dos tokens JWT foram delegadas a um componente específico de identidade, sem incorporar essa responsabilidade diretamente à lógica principal do sistema.	Aumenta a organização da segurança, reduz o acoplamento no backend e melhora o controle de autenticação e acesso.

PostgreSQL como persistência relacional	O domínio financeiro do sistema envolve relações entre usuários, importações, registros financeiros, históricos e auditoria, exigindo consistência e integridade transacional.	Aumenta a confiabilidade das consultas e a consistência dos dados persistidos.
CSV como formato inicial de entrada	O MVP depende da importação de arquivos financeiros estruturados, e o uso de CSV simplifica a entrada de dados nesta etapa inicial do projeto.	Viabiliza a ingestão de informações com menor complexidade de implementação e menor esforço de integração externa.
Padronização do ambiente com Docker	O projeto envolve diferentes componentes e dependências que precisam ser executados de forma consistente entre os membros da equipe e no contexto acadêmico.	Facilita a configuração do ambiente, reduz inconsistências entre máquinas e melhora a reprodutibilidade da solução.

7. CONCLUSÃO

A arquitetura definida para o CashLens foi estruturada para atender às necessidades do MVP com clareza técnica, separação de responsabilidades e base adequada para evolução futura. A adoção do modelo cliente-servidor, associada à organização em camadas, ao uso de React na interface, Spring Boot no backend, Keycloak na autenticação e PostgreSQL na persistência, estabelece uma solução consistente para suportar cadastro e acesso de usuários, ingestão de arquivos CSV, processamento das regras de negócio e visualização analítica dos dados financeiros.

Essa definição arquitetural contribui para a manutenção do sistema, para a compreensão dos componentes pelos diferentes envolvidos no projeto e para a implementação incremental das funcionalidades previstas na disciplina. Além disso, a separação entre apresentação, identidade e acesso, aplicação, negócio e dados reduz o acoplamento entre as partes da solução e favorece sua ampliação de forma mais organizada.

Este documento poderá ser refinado conforme o amadurecimento do projeto, especialmente com a evolução das regras de negócio, a ampliação dos mecanismos de segurança, a consolidação dos endpoints do sistema e a inclusão de novas integrações previstas para versões futuras.